Mathematical Foundations of the Ultimate Teacher Model

ResNet-50 on STL-10 with Pseudo-Labeling, CutMix, and Multi-GPU Data Parallelism

Shrink or Sink 2025

March 2026

Contents

1 Overview of the Training Pipeline **2**

2 The ResNet-50 Architecture **2**
2.1 Residual Block 2
2.2 Full Network Composition 2

3 Phase A: Supervised Burn-in (Epochs 1–50) **3**
3.1 Loss Function 3
3.2 CutMix Augmentation 3

4 Phase B: Pseudo-Label Generation (Epoch 51) **3**

5 Phase C: Mastery Training (Epochs 51–200) **4**
5.1 Optimizer: AdamW 4
5.2 Cosine Annealing LR Schedule 4
5.3 Gradient Clipping 5

6 Multi-GPU Data Parallelism **5**
6.1 Mathematical Formulation 5
6.2 Implementation Note: State Dict Keys 5

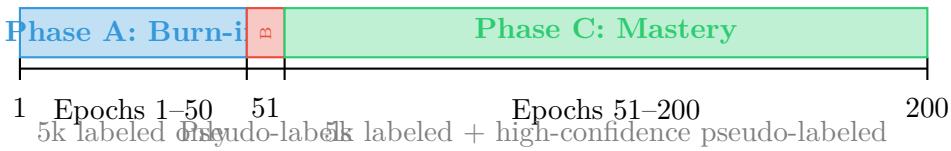
7 Expected Validation Accuracy Trajectory **6**

8 Full Training Algorithm **7**

1 Overview of the Training Pipeline

The teacher is a **ResNet-50** trained in a three-phase curriculum over $N = 200$ total epochs on the STL-10 dataset:

$$N = N_{\text{burn-in}} + N_{\text{mastery}} = 50 + 150$$



2 The ResNet-50 Architecture

2.1 Residual Block

The fundamental operation of ResNet-50 is the *bottleneck residual block*. Given input $\mathbf{x}$, the output is:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x} \tag{1}$$

where $\mathcal{F}$ is the residual function, a composition of three convolutions:

$$\mathcal{F}(\mathbf{x}) = W_3 * \sigma(\text{BN}(W_2 * \sigma(\text{BN}(W_1 * \mathbf{x})))) \tag{2}$$

The channel dimensions for each bottleneck block follow the pattern $C \to C/4 \to C/4 \to C$ for the 1×1, 3×3, and 1×1 convolutions respectively, achieving a *4× channel reduction* inside the block.

When input/output dimensions differ (e.g., when stride > 1), a learned shortcut projection W_s is used:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + W_s\mathbf{x} \tag{3}$$

2.2 Full Network Composition

ResNet-50 applies $L = 4$ macro-stages, each containing $[3, 4, 6, 3]$ bottleneck blocks. For an STL-10 image $\mathbf{x}_0 \in \mathbb{R}^{3\times96\times96}$:

$$h_0 = \text{MaxPool}(\sigma(\text{BN}(W_{\text{stem}} * \mathbf{x}_0))) \in \mathbb{R}^{64\times24\times24} \tag{4}$$

$$h_1 = \text{Stage}_1(h_0) \in \mathbb{R}^{256\times24\times24} \tag{5}$$
$$h_2 = \text{Stage}_2(h_1) \in \mathbb{R}^{512\times12\times12} \tag{6}$$
$$h_3 = \text{Stage}_3(h_2) \in \mathbb{R}^{1024\times6\times6} \tag{7}$$
$$h_4 = \text{Stage}_4(h_3) \in \mathbb{R}^{2048\times3\times3} \tag{8}$$

The final representation is obtained via global average pooling and the linear head:

$$\mathbf{z} = W_{\text{fc}}\,\text{GAP}(h_4) \in \mathbb{R}^{10} \qquad \text{(10 classes for STL-10)} \tag{9}$$

3 Phase A: Supervised Burn-in (Epochs 1–50)

During the burn-in, the teacher trains on the 5,000 labeled examples $\mathcal{D}_L = \{(\mathbf{x}_i, y_i)\}_{i=1}^{5000}$.

3.1 Loss Function

Label-smoothed Cross Entropy with smoothing parameter $\varepsilon = 0.1$ and $K = 10$ classes:

$$\mathcal{L}_{\text{LS}}(\mathbf{z}, y) = -\sum_{k=1}^{K} \tilde{p}(k|y) \log \hat{p}_k \tag{10}$$

where the smoothed target distribution is:

$$\tilde{p}(k|y) = (1-\varepsilon)\,\mathbf{1}[k = y] + \frac{\varepsilon}{K} \tag{11}$$

and the predicted distribution is $\hat{p}_k = \text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$.

3.2 CutMix Augmentation

With probability 0.5, CutMix is applied. Given two samples $(\mathbf{x}_a, y_a)$ and $(\mathbf{x}_b, y_b)$, a rectangular mask M is sampled and:

$$\tilde{\mathbf{x}} = M \odot \mathbf{x}_a + (1 - M) \odot \mathbf{x}_b \tag{12}$$

The mixing ratio λ is adjusted for the actual area of the cut box:

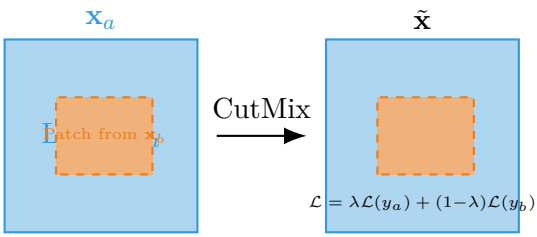
$$\lambda = 1 - \frac{(x_2 - x_1)(y_2 - y_1)}{W \cdot H} \tag{13}$$

The crop box coordinates are sampled from a **Beta distribution**:

$$\lambda_0 \sim \text{Beta}(\alpha, \alpha), \quad \alpha = 1.0 \qquad r_W = W\sqrt{1-\lambda_0},\ r_H = H\sqrt{1-\lambda_0} \tag{14}$$

The mixed loss becomes:

$$\mathcal{L}_{\text{CutMix}} = \lambda\,\mathcal{L}(\mathbf{z}, y_a) + (1-\lambda)\,\mathcal{L}(\mathbf{z}, y_b) \tag{15}$$



4 Phase B: Pseudo-Label Generation (Epoch 51)

At the start of epoch 51, the teacher (frozen) performs inference on the 100,000 unlabeled images $\mathcal{D}_U = \{\mathbf{u}_i\}_{i=1}^{100000}$:

$$\hat{p}_k^{(i)} = \text{softmax}(f_\theta(\mathbf{u}_i))_k \tag{16}$$

$$(\hat{c}^{(i)}, \hat{\pi}^{(i)}) = \left(\arg\max_k \hat{p}_k^{(i)},\ \max_k \hat{p}_k^{(i)} \right) \tag{17}$$

Only images with confidence above threshold τ_s are retained:

$$\mathcal{D}_P = \left\{ (\mathbf{u}_i, \hat{c}^{(i)}) \;\middle|\; \hat{\pi}^{(i)} > \tau_s \right\}, \quad \tau_s = 0.85 \tag{18}$$

Because we use a strict threshold, we heavily filter the unlabeled set, retaining only images the model is highly certain about. The expected precision of pseudo-labels under threshold τ_s is high:

$$\mathbb{P}[\hat{c}^{(i)} = y_i \mid \hat{\pi}^{(i)} > \tau_s] \gg \frac{1}{K} \tag{19}$$

5 Phase C: Mastery Training (Epochs 51–200)

The combined dataset is:

$$\mathcal{D}_C = \mathcal{D}_L \cup \mathcal{D}_P \tag{20}$$

Training continues from epoch 51 to 200 on $\mathcal{D}_C$ using the identical loss (Eq. 10 + CutMix).

5.1 Optimizer: AdamW

$$\theta_{t+1} = \theta_t - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \eta_t \lambda_{\text{wd}}\, \theta_t \tag{21}$$

where the bias-corrected moment estimates are:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \tag{22}$$

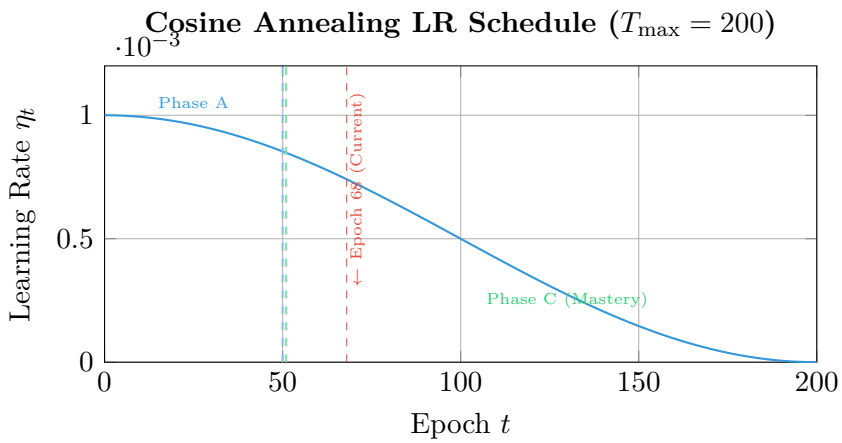
$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \tag{23}$$

with $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, base $\eta_0 = 10^{-3}$, $\lambda_{\text{wd}} = 10^{-4}$.

5.2 Cosine Annealing LR Schedule

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min}) \left(1 + \cos \frac{\pi t}{T_{\max}} \right) \tag{24}$$

with $T_{\max} = 200$, $\eta_{\min} = 0$.



5.3 Gradient Clipping

To prevent gradient explosions when training on noisy pseudo-labels, gradient norms are clipped:

$$g_t \leftarrow g_t \cdot \min\left(1,\ \frac{G_{\max}}{\|g_t\|_2}\right), \quad G_{\max} = 1.0 \tag{25}$$

6 Multi-GPU Data Parallelism

6.1 Mathematical Formulation

With $P = 2$ GPUs (T4 $\times$ 2), a mini-batch $\mathcal{B}$ of size $B = 128$ is partitioned:

$$\mathcal{B} = \mathcal{B}_1 \cup \mathcal{B}_2, \quad |\mathcal{B}_p| = B/P = 64 \tag{26}$$

Each GPU p holds a *replica* of the model $f_{\theta^{(p)}}$ and computes its own loss:

$$\mathcal{L}_p = \frac{1}{|\mathcal{B}_p|} \sum_{(\mathbf{x},y)\in\mathcal{B}_p} \ell(f_{\theta^{(p)}}(\mathbf{x}), y) \tag{27}$$

Gradients are computed independently on each device:

$$g_p = \nabla_{\theta^{(p)}} \mathcal{L}_p \tag{28}$$

After the backward pass, **gradients are averaged and aggregated (AllReduce)** on GPU 0:

$$\bar{g} = \frac{1}{P} \sum_{p=1}^{P} g_p \tag{29}$$

The optimizer step runs once on GPU 0 using $\bar{g}$ and updated weights θ^* are **broadcast** back to all replicas before the next iteration:

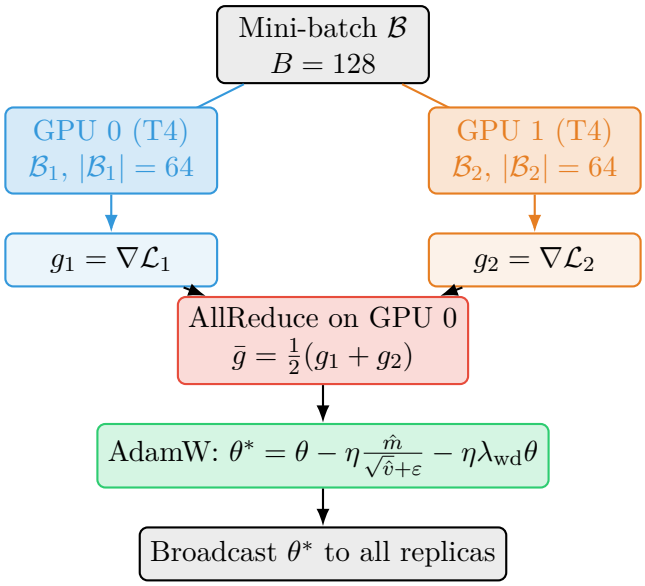
$$\theta^{(p)} \leftarrow \theta^* = \mathrm{AdamW}(\theta_0,\ \bar{g}), \quad \forall p \tag{30}$$

This is mathematically equivalent to training on the full batch $\mathcal{B}$ on a single GPU, but each forward/backward pass takes roughly $\frac{1}{P}$ of the time.

6.2 Implementation Note: State Dict Keys

`nn.DataParallel` normally prepends `module.` to every key in the state dict. To preserve checkpoint portability, the training script explicitly strips this prefix before saving:

$$\texttt{save_state} = \theta^*.\mathit{state_dict}() \quad (\text{via } \texttt{model.module.state_dict()} \text{ when wrapped}) \tag{31}$$



7 Expected Validation Accuracy Trajectory

The following figure illustrates the expected qualitative accuracy curve from the combination of pseudo-labeling and cosine annealing, along with the current checkpoint state.

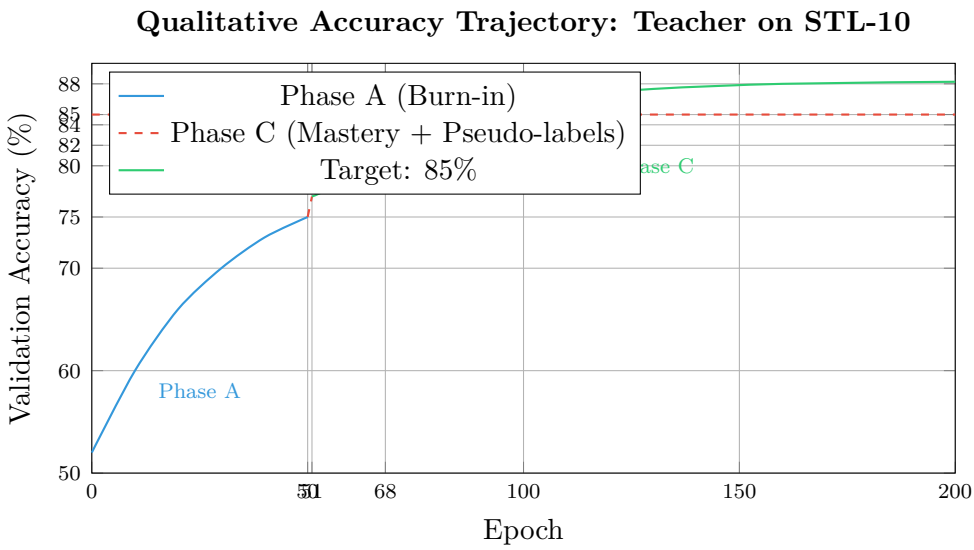


Table 1: Summary of training hyperparameters for the Ultimate Teacher Model.

Hyperparameter	**Symbol**	**Value**
Base learning rate	η_0	1×10^{-3}
Weight decay	λ_{wd}	1×10^{-4}
Adam β_1	β_1	0.9
Adam β_2	β_2	0.999
LR schedule	—	Cosine Annealing
$T_{\max}$	$T_{\max}$	200
Label smoothing	ε	0.1
CutMix probability	—	0.5
CutMix α	α	1.0
Pseudo-label threshold	τ_s	0.85
Max gradient norm	$G_{\max}$	1.0
Batch size (total)	B	128
Batch size per GPU	B/P	64
# GPUs	P	2 (T4 $\times$ 2)

8 Full Training Algorithm

Algorithm 1 Ultimate Teacher Training Pipeline

Initialize $\theta \sim \mathcal{N}(0, \sigma^2)$ (or warm-restart weights)
Load checkpoint if available; set $t \leftarrow t_{\text{resume}}$
Initialize replicas $\theta^{(p)} \leftarrow \theta$ on each GPU p
for $t = 1$ to $N = 200$ **do**
  if $t = N_{\text{burn-in}} + 1$ **and** $\mathcal{D}_P = \emptyset$ **then**
    Phase B: Run inference on $\mathcal{D}_U$; collect $\mathcal{D}_P = \{(\mathbf{u}, \hat{c}) : \hat{\pi} > \tau_s\}$
  end if
  $\mathcal{D}_{\text{active}} \leftarrow \mathcal{D}_L$ if $t \leq N_{\text{burn-in}}$ else $\mathcal{D}_L \cup \mathcal{D}_P$
  for each mini-batch $\mathcal{B} \sim \mathcal{D}_{\text{active}}$ **do**
    Scatter $\mathcal{B}$ to GPUs: $\mathcal{B}_p = \mathcal{B}[(p-1)B/P : pB/P]$
    for each GPU p **in parallel do**
      if `rand()` > 0.5 **then** apply CutMix to $\mathcal{B}_p$
      end if
      $\mathcal{L}_p \leftarrow$ label-smoothed CE (+ CutMix mixture) on $\mathcal{B}_p$
      $g_p \leftarrow \nabla_\theta \mathcal{L}_p$
    end for
    $\bar{g} \leftarrow \frac{1}{P} \sum_p g_p$ (AllReduce)
    Clip: $\bar{g} \leftarrow \bar{g} \cdot \min(1, G_{\max}/\|\bar{g}\|_2)$
    $\theta \leftarrow \text{AdamW}(\theta, \bar{g})$; broadcast θ to all $\theta^{(p)}$
  end for
  $\eta_t \leftarrow \frac{\eta_0}{2}\left(1 + \cos\frac{\pi t}{T_{\max}}\right)$ (Cosine Annealing)
  Evaluate on validation set; save best and latest checkpoint
end for
return θ^* (best validation accuracy weights)